



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

**ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №7
«РАБОТА С ВРЕМЕННЫМИ
РЯДАМИ»**

по дисциплине «Языки программирования для статистической обработки
данных»

Студент группы

ИНБО-20-23. Боргачев Т.М.

(подпись)

Преподаватель

Трушин СМ

(подпись)

Москва 2025 г.

ВВЕДЕНИЕ

Цель практики: научиться анализировать временные ряды, выявлять тренды и сезонность, а также использовать визуальные инструменты для их интерпретации в Python, R.

Задачи практики:

1. Выполнить анализ временных рядов:

- Загрузить данные временных рядов в Python, R.
- Выявить тренды и сезонность.
- Python: использование pandas (rolling, resample).
- R: использование пакетов forecast, zoo.

2. Выполнить визуализацию временных рядов:

- Построение графиков трендов, сезонности и остатков:
- Python: matplotlib и seaborn.
- R: ggplot2 и forecast.

3. Сравнить результаты анализа и визуализации в Python, R

4. Определить, какие инструменты наиболее подходят для анализа временных рядов в зависимости от задач.

Теоретическое введение

РЕЗУЛЬТАТЫ РАБОТЫ

Подготовка данных

Предварительная обработка данных заключается в выборе числовых данных, обработке пропусков и преобразования даты в требуемый формат.

Код для предобработки данных на R представлен в листинге 1.

Листинг 1 – Предобработка данных для временных рядов в R

```
library(readr)
library(ggcorrplot)
df <- read_csv("train_time_series.csv")
head(df)
library(dplyr)
numeric_df <- df %>% select(where(is.numeric))
df$date <- as.Date(df$date, format = "%Y-%m-%d")
str(df)
df_ts <- ts(df$sales, start=c(2020,1), frequency=365)
numeric_df <- numeric_df %>%
  mutate(across(everything(), ~ ifelse(is.na(.), mean(., na.rm = TRUE), .)))
```

Код для предобработки данных на Python представлен в листинге 2.

Листинг 2 – Предобработка данных для временных рядов в Python

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv("train_time_series.csv")
df.head()
from statsmodels.tsa.seasonal import seasonal_decompose
df['date'] = pd.to_datetime(df['date'])
df.set_index('date', inplace=True)
df['sales'] = pd.to_numeric(df['sales'], errors='coerce')
df['sales'].fillna(method='ffill', inplace=True)
df['rolling_mean_7'] = df['sales'].rolling(window=7).mean()
```

Анализ временных рядов

Код для анализа временных рядов на R представлен в листинге 3.

Листинг 3 – Анализ временных рядов в R

```
# df_ts (для ежедневных данных, к примеру)
df_ts <- ts(df$sales, start=c(2020,1), frequency=365)
decomp <- decompose(df_ts, type="additive")

plot(decomp)

library(forecast)
# Автоматическая декомпозиция STL
fit <- stl(df_ts, s.window="periodic")
plot(fit)
```

Код для анализа временных рядов на Python представлен в листинге 4.

Листинг 4 – Анализ временных рядов в Python

```
plt.figure(figsize=(12, 6))
plt.plot(df.index, df['sales'], label='Исходный ряд', color='blue', alpha=0.7)
plt.plot(df.index, df['rolling_mean_7'], label='7-дневное скользящее среднее',
color='red', linestyle='--')
plt.title("Анализ временного ряда (тренд и сглаживание)", fontsize=16)
plt.xlabel("Дата", fontsize=14)
plt.ylabel("Значение", fontsize=14)
plt.grid(alpha=0.3)
plt.legend(fontsize=12)
plt.tight_layout()
plt.show()

decomposition = seasonal_decompose(df['sales'], model='additive', period=30)
decomposition.plot()
plt.show()
```

Результат построения временного ряда на R представлен на рис. 1.

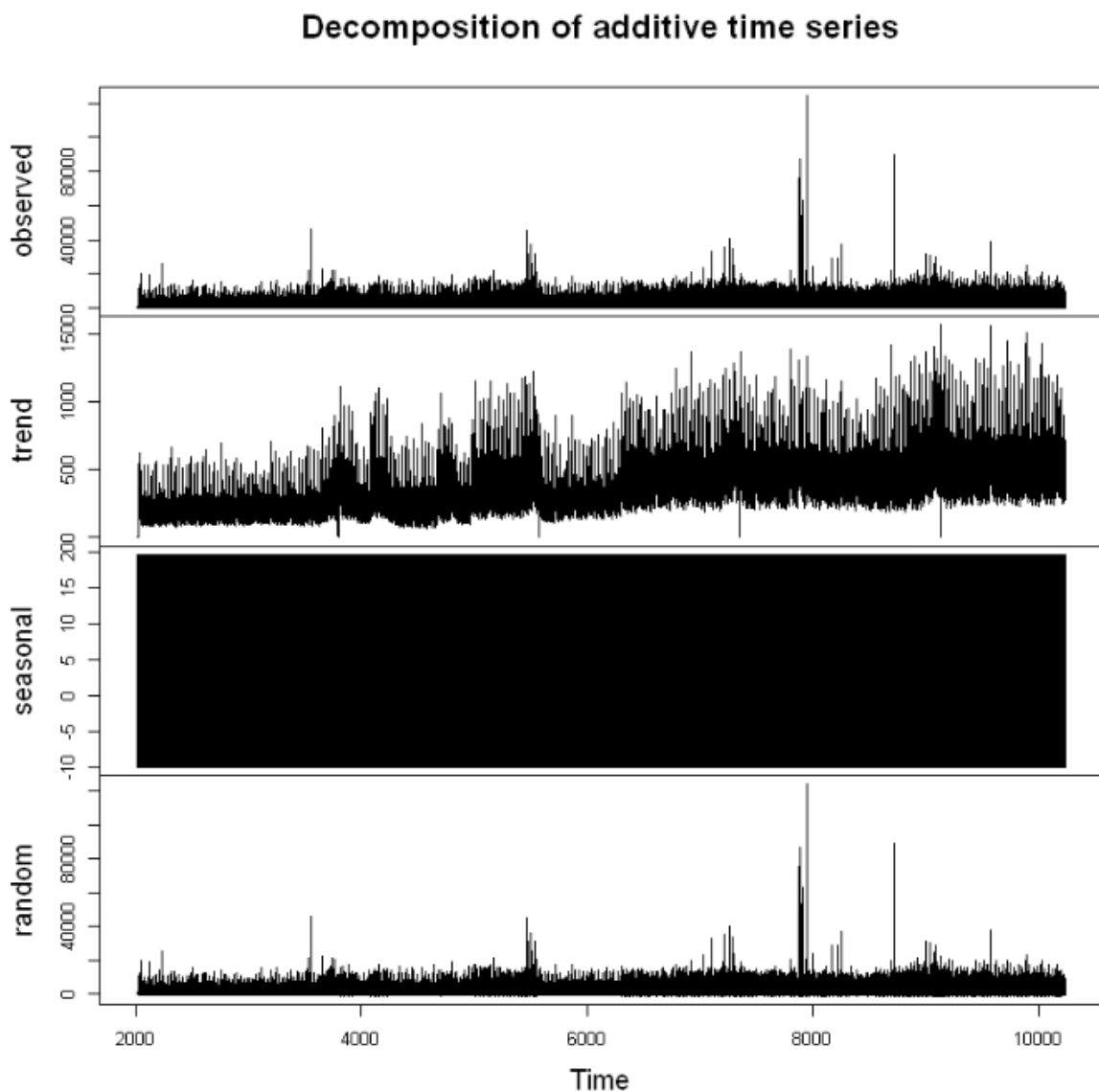


Рисунок 1 – Декомпозиция в R

Результаты построения временного ряда на Python представлены на рис. 2 и рис. 3.



Рисунок 2 – Тренд и сглаживание в Python

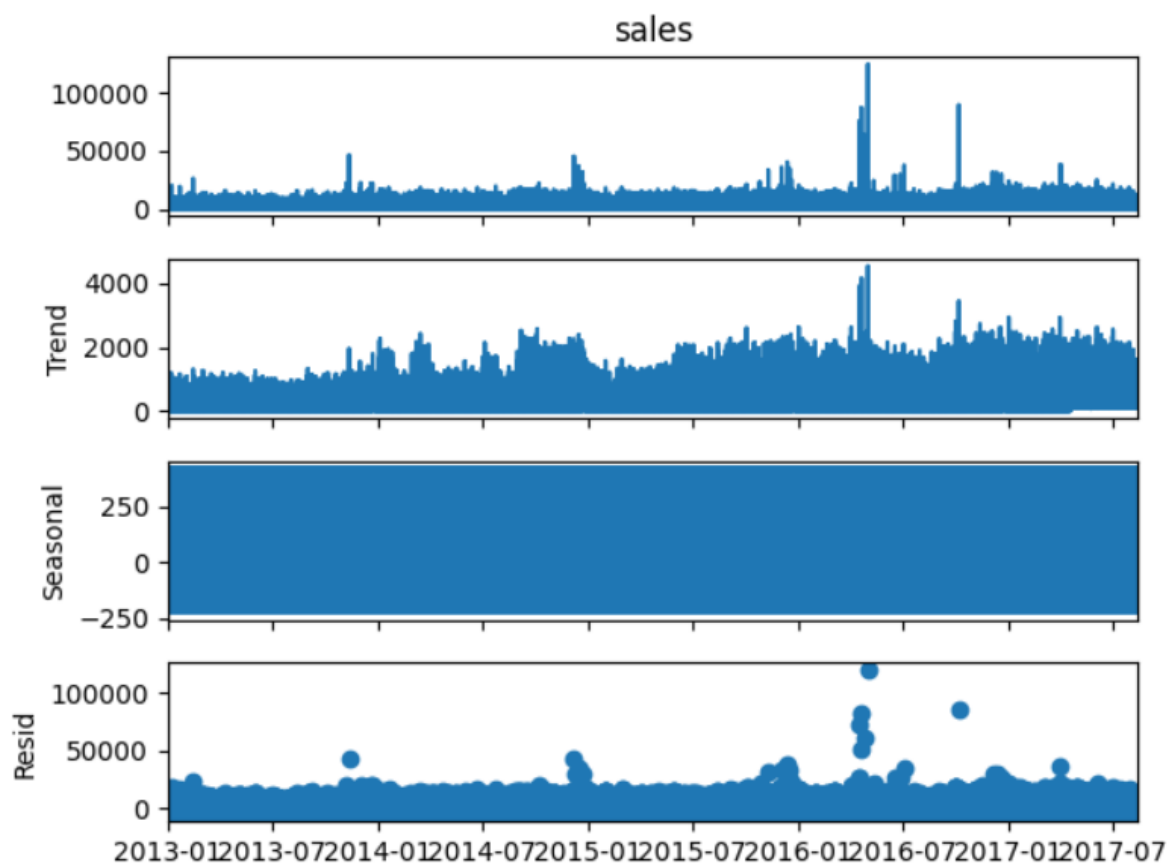


Рисунок 3 – Декомпозиция в Python

Исходный ряд имеет значительную волатильность, существуют периоды с высокими значениями и периоды с низкими значениями. Красная линия представляет собой 7-дневное скользящее среднее. Она более гладкая по сравнению с исходным рядом и помогает уменьшить шум и волатильность. На данном графике видно, что есть периоды роста и падения, но нет четко выраженного долгосрочного тренда.

По декомпозиции временного ряда, который был разбит на три компонента:

наблюдаемое значение, тренд и сезонные колебания – рассмотрим каждый из этих компонентов подробнее:

1. Наблюдаемое значение (top panel):

- Это оригинальный временной ряд, который содержит все элементы: тренд, сезонные колебания и случайные шумы.
- Видно, что ряд имеет значительную волатильность и периодические колебания.

2. Тренд (middle panel):

- Тренд отражает общую динамику изменения временного ряда за время.
- В данном случае тренд относительно стабилен, без явных признаков роста или спада.
- Он может быть полезен для определения основной тенденции, если нужно удалить сезонные и шумовые компоненты.

3. Сезонные колебания (bottom panel):

- Сезонные колебания представляют повторяющиеся циклы, связанные со временем года или другими регулярными факторами.
- В данном случае сезонные колебания имеют периодическую структуру, но их конкретные характеристики трудно определить из этого графика.

4. Случайные шумы (not shown explicitly but implied by the other components):

- Случайные шумы являются остатками после удаления тренда и сезонных колебаний.
- Они отвечают за непредсказуемые изменения, которые остаются после обработки данных.

Визуализация временных рядов

Код для визуализации временных рядов данных на R представлен в листинге 5.

Листинг 5 – Визуализация ряда в R

```
ggplot(df, aes(x=date, y=sales)) +  
  geom_line() +  
  ggtitle("Временной ряд") +  
  xlab("Дата") + ylab("Значение")  
df$Month <- format(df$date, "%m")  
ggplot(df, aes(x=Month, y=sales)) +  
  geom_boxplot() +  
  ggtitle("Сезонность по месяцам") +  
  xlab("Месяц") + ylab("Значение")
```

Код для визуализации временных рядов на Python представлен в листинге 6.

Листинг 6 – Визуализация ряда в Python

```
plt.figure(figsize=(10, 5))
plt.plot(df.index, df['sales'], marker='o')
plt.title("Временной ряд")
plt.xlabel("Дата")
plt.ylabel("Значение")
plt.show()
df['Month'] = df.index.month
sns.boxplot(x='Month', y='sales', data=df)
plt.title("Сезонность по месяцам")
plt.show()
```

Результаты визуализации ряда на R представлены на рис. 4 и рис. 5.

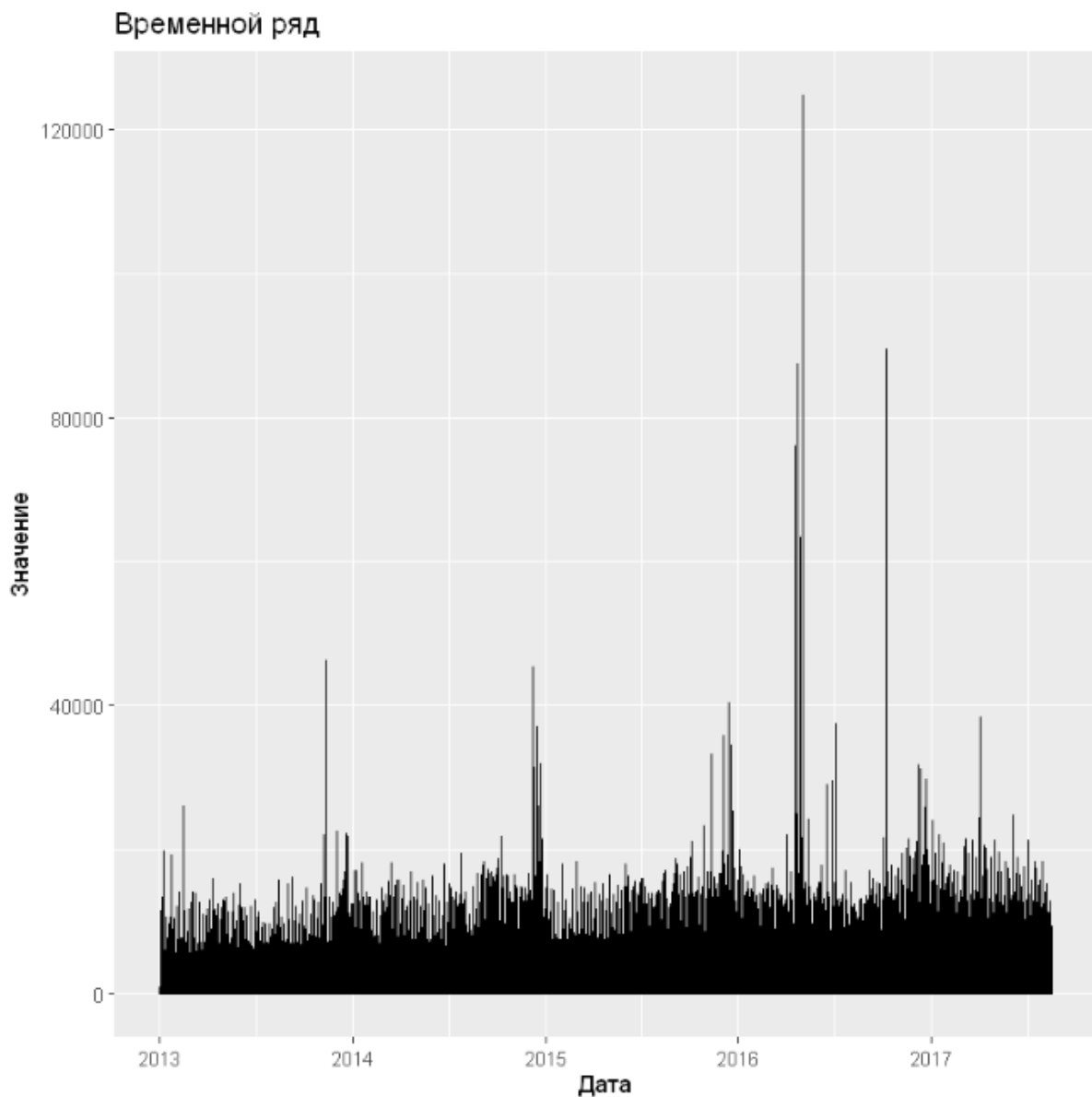


Рисунок 4 – Визуализация ряда в R

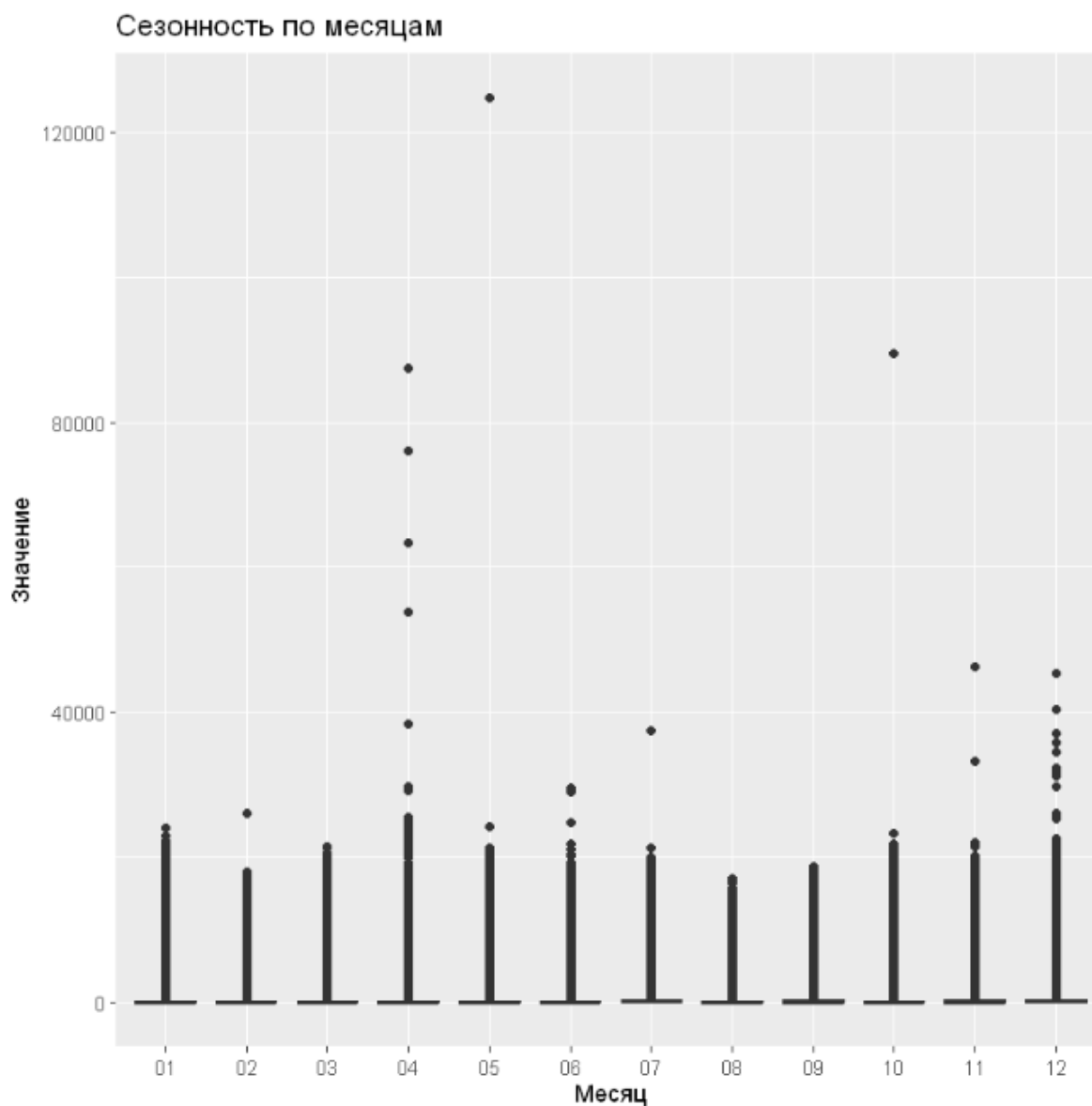


Рисунок 5 – Визуализация сезонности в R

Результаты визуализации ряда на Python представлены на рис. 6 и рис. 7.

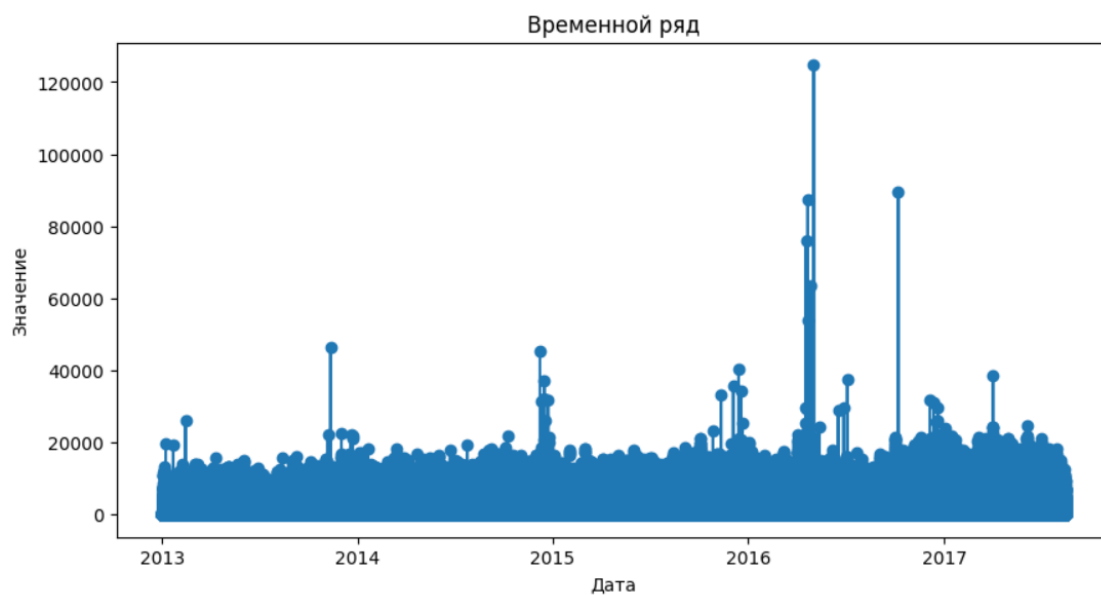


Рисунок 6 – Визуализация ряда в Python

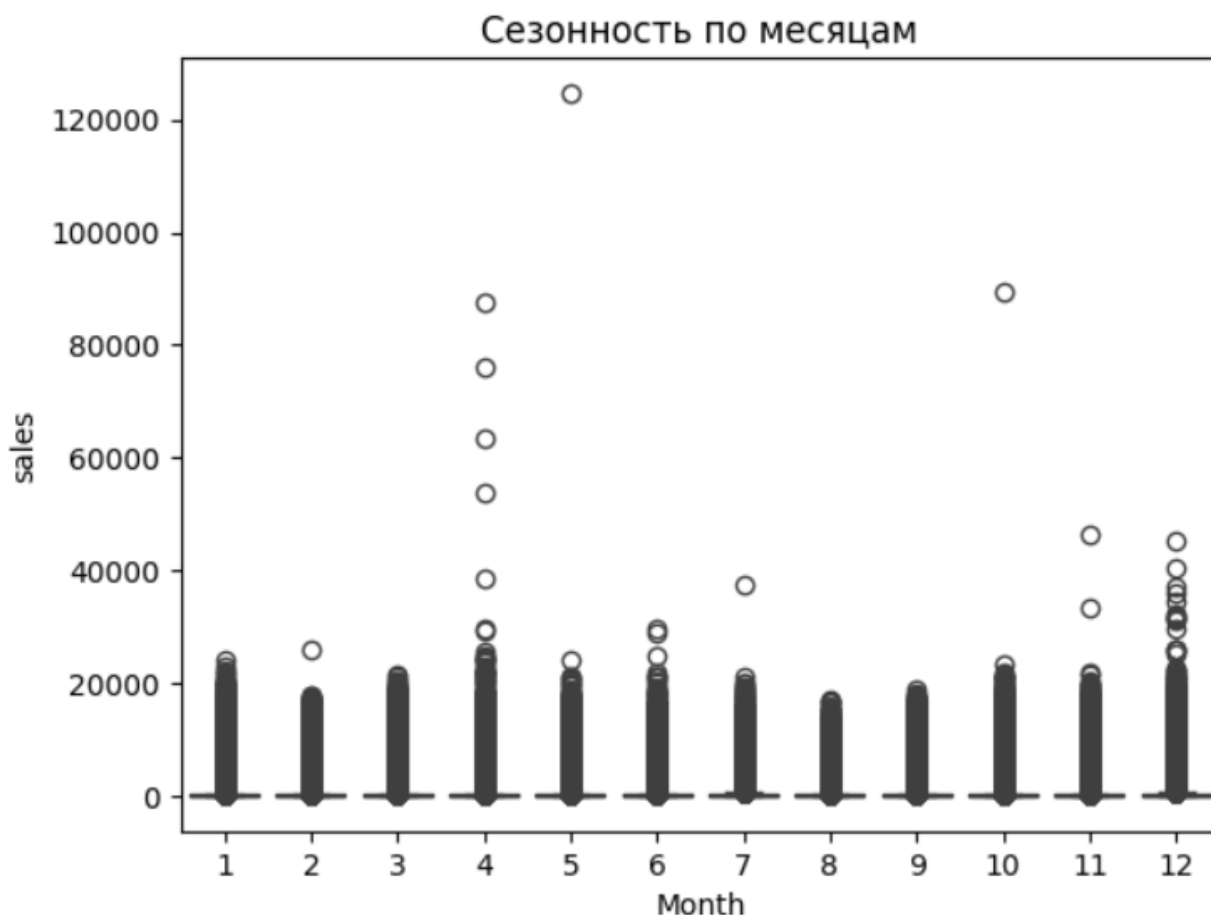


Рисунок 7 – Сезонность в Python

Сравнение результатов

В ходе анализа временных рядов было проведено несколько шагов, включая создание графиков и декомпозицию временного ряда. Результаты были представлены на различных платформах для анализа данных.

Графики анализа временного ряда

На Python были созданы следующие графики:

- Анализ временного ряда с использованием тренда и сглаживания.
- Декомпозиция временного ряда на наблюдаемое значение, тренд, сезонные колебания и случайные шумы.

На R были выполнены аналогичные операции, однако графики в R оказались более наглядными и удобными для визуализации. В частности, графики сезонности и декомпозиции временного ряда на R выглядели более чистыми и легче интерпретировать.

Визуализация временного ряда

На Python была создана визуализация временного ряда с использованием библиотеки Matplotlib. График был построен на основе исходных данных, где отображены значения временного ряда по времени.

На R была выполнена аналогичная операция, но графики в R были более детализированными и удобными для анализа. Визуализация временного ряда на R

позволила более точно определить ключевые моменты и тенденции в данных.

Декомпозиция временного ряда

Декомпозиция временного ряда на Python была выполнена с помощью библиотеки statsmodels. Результаты были представлены в виде графиков, где отдельно показывались тренд, сезонные колебания и случайные шумы.

На R была выполнена аналогичная операция, но графики в R оказались более ясными и удобными для анализа. Декомпозиция временного ряда на R позволила более точно определить составляющие компоненты временного ряда.

ЗАКЛЮЧЕНИЕ

В рамках практической работы были изучены и применены методы анализа временных рядов с использованием языков программирования Python и R. Были выполнены загрузка, предобработка данных, выявление трендов и сезонности, а также визуализация результатов.

В ходе работы установлено, что оба языка программирования имеют мощные инструменты для анализа временных рядов, но с различной спецификой применения:

- Python показал себя эффективным для быстрого анализа данных благодаря библиотекам `pandas` и `matplotlib`, особенно удобен для инженерных расчетов и машинного обучения
- R продемонстрировал более продвинутые возможности визуализации и статистического анализа временных рядов благодаря специализированным пакетам `forecast` и `ggplot2`

По результатам сравнения графиков и декомпозиции временных рядов можно сделать вывод, что R предоставляет более наглядные и информативные визуализации, что особенно важно при детальном анализе сезонности и трендов. Однако Python остается более универсальным инструментом, подходящим как для анализа данных, так и для разработки комплексных решений.

Полученные навыки позволяют выбирать оптимальный инструмент в зависимости от поставленной задачи: для быстрого анализа и интеграции с другими системами предпочтителен Python, а для глубокого статистического анализа и качественной визуализации – R.